

2 - Il paradigma Orientato agli Oggetti

Introduzione

Un difetto fondamentale della programmazione imperativa è che **le variabili globali sono potenzialmente accessibili da ogni parte del programma**. I grandi programmi che permettono l'accesso alle variabili globali tendono ad essere ingestibili, la ragione è che nessun modulo che accede ad una variabile globale può essere sviluppato e compreso indipendentemente da altri moduli che pure accedono alla medesima variabile

Il rimedio fu ovviamente **l'information hiding**, ossia quello di incapsulare in un modulo ogni variabile globale insieme a un gruppo di operazioni autorizzate ad accedervi, gli altri moduli non possono accedervi direttamente ma indirettamente chiamando queste operazioni. Il loro nome era proprio quello di **oggetti**

In alcuni linguaggi operativi era già possibile definire degli **oggetti**, ma:

- Il concetto dell'oggetto non fa parte della base del linguaggio, il programmatore deve controllare come venga utilizzati
- Quando sono introdotti, non sono trattati come **cittadini di prima classe**, ossia non avevano tutte le funzionalità previste dal paradigma

Paradigma Object Oriented

Confrontando il paradigma orientato a oggetti rispetto a quello imperativo si può dire che esso costituisce:

- Una **evoluzione**, in quanto permette agli oggetti di essere cittadini di prima classe
- Una **rivoluzione**, in quanto gli oggetti assumono un ruolo fondamentale nella progettazione e nella programmazione. Ora l'information hiding e l'incapsulamento sono principi cardine nel paradigma orientato a oggetti

Classificazione di Wegner

I linguaggi di programmazione si classificano in:

- **Object-based**: supportano la nozione di oggetto (come il linguaggio Modula-2), qui non sono ancora trattati come cittadini di prima classe
- **Class-based**: supportano la nozione di oggetto e classe (come il linguaggio Ada-83)
- **Object-oriented**: supportano la nozione di oggetto, classe, ereditarietà (Smalltalk, C++, Java, ...)

 **Domanda di esame: Quali sono le caratteristiche di un linguaggio ad oggetti?**

Gli oggetti

Gli oggetti quindi sono questo particolare costrutto che permettono di incapsulare **uno stato e un comportamento**:

- Lo **stato** è identificato dal contenuto di una certa area di memoria (quindi dalle variabili)
- Il **comportamento** è definito da una collezione di procedure e funzioni (chiamate **metodi**) che possono operare sulla rappresentazione dell'area di memoria associata all'oggetto.

Un oggetto implementato bene quindi permette la modifica delle variabili soltanto tramite delle sue funzioni dedicate

☰ Esempio di oggetto

Esempio: in un gioco elettronico che usa una palla, si può pensare alla palla come un oggetto dotato di uno

- **Stato**: dimensione, posizione in uno spazio di riferimento, etc.
- **Comportamento**: una palla può 'apparire' o 'scompare' dallo schermo, può muoversi, può rimbalzare, etc.

Le variabili e i metodi definiti nell'oggetto 'palla' stabiliscono lo stato e il comportamento che sono rilevanti all'uso della palla nel gioco elettronico

Identificatore di oggetto

Un oggetto ha la sua **identità**, cioè è riconoscibile indipendentemente dal suo stato corrente. Per questo ogni oggetto ha un identificatore di oggetto (object identifier, OID) che lo identifica univocamente. In alcuni contesti gli OID sono anche detti **riferimenti (references)** (in alcuni linguaggi prende questo nome poichè coincide con l'indirizzo di memoria con l'oggetto utilizzato)

Un identificatore di oggetto è **immutabile**, cioè non può essere modificato da una qualche opzione di programmazione. Cambiare l'OID di un oggetto equivale alla cancellazione dell'oggetto e alla creazione di un altro oggetto con lo stesso stato.

Il concetto di uguaglianza non è uguale a quello di identificatore, infatti si può avere lo stesso stato di due oggetti con due OID differenti.

Normalmente gli OID sono assegnati in modo automatico agli oggetti, sicché non hanno un significato nel mondo reale. In molti ambienti di programmazione object-oriented, l'OID corrisponde all'indirizzo dell'area di memoria che conserva lo stato dell'oggetto. Quasi mai il programmatore utilizza esplicitamente i riferimenti, generalmente questi vengono legati a delle variabili e si fa riferimento agli oggetti mediante gli identificatori di variabile, ma variabili distinte potrebbero far riferimento al medesimo oggetto, in questo caso si hanno degli **alias**.

La presenza di alias non significa che un oggetto non è identificato univocamente, ma semplicemente che **diversi identificatori di variabile sono stati legati al medesimo riferimento di oggetto**.

Lo stato di un oggetto può anche contenere il riferimento ad un altro oggetto, si dice quindi che un **oggetto punta ad un altro**. Il puntamento è asimmetrico, ossia questo puntamento va in unica direzione.

UML

L'UML è un linguaggio visuale utilizzato per

- Definire
- Progettare
- Realizzare
- Documentare

È un linguaggio di **rappresentazioni dei sistemi universale** (sistemi eterogenei per architettura, tecnologie, tipologia applicativa). L'UML è di supporto sia alla progettazione di un nuovo sistema che alla documentazione di un software esistente.

Essendo molto versatile può essere anche utilizzato in molte fasi del ciclo di vita del software (come nelle relazioni clienti-fornitori o ingegnere-ingegnere) senza scendere nei dettagli di un linguaggio di programmazione.

La notazione, sintassi e semantica sono standard secondo le specifiche UML.

L'UML permette di usare una notazione universale per illustrare alcuni concetti del paradigma orientato a oggetti, senza fare così riferimento a un particolare linguaggio di programmazione.

Ogni simbolo in UML ha una **semantica ben definita** che ne permette l'**univoca interpretazione** fra diversi interlocutori e strumenti software

Modellazione Object-Oriented

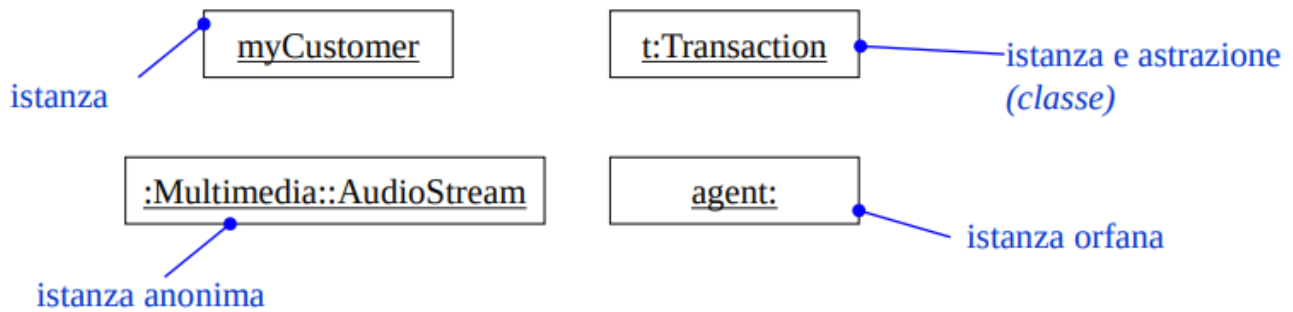
Nella modellazione object-oriented le componenti elementari sono l'**oggetto** e la **classe**:

- Un oggetto è qualcosa tratta generalmente dal vocabolario dello spazio del problema o dello spazio della soluzione. Ogni oggetto ha un'identità, stato e comportamento.
- Una classe è una descrizione di un insieme di oggetti omogenei

Visualizzare, specificare, costruire, e documentare sistemi object-oriented è esattamente lo scopo dello UML

Oggetti (Istanze in UML)

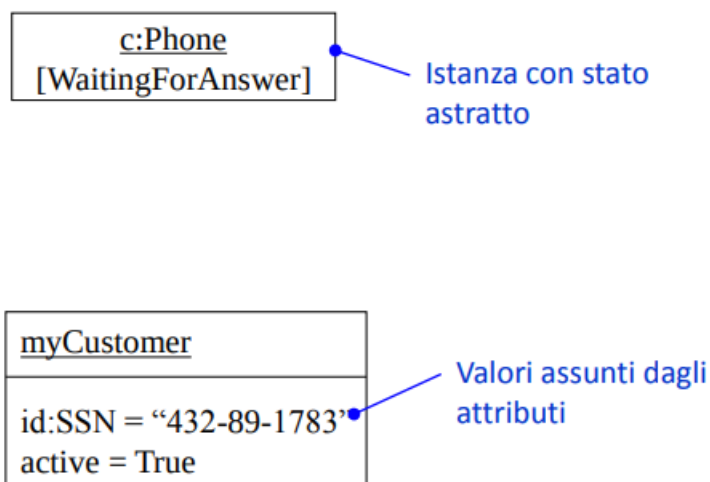
In UML un oggetto (o istanza) è graficamente rappresentato in questo modo:



Quindi è possibile indicare:

- Solo il nome (identificatore) dell'istanza
- Nome dell'istanza e dell'astrazione (la classe) a cui appartiene
- Solo il nome dell'astrazione, qualora non si conosca a priori il nome dell'istanza
- Un'istanza orfana, se non si conosce a priori la sua astrazione

Lo stato di un oggetto può essere rappresentato in modo astratto, oppure indicando esplicitamente i valori assunti dagli attributi dell'oggetto



Classi

Una classe è la descrizione di una famiglia di oggetti che condividono la stessa struttura (gli attributi) e il medesimo comportamento (operazioni).

Nella programmazione OO ogni oggetto è un'istanza di una classe, cioè un oggetto non può essere ottenuto se non si definisce la sua classe di appartenenza. Analogamente nella modellazione OO le istanze **esistono in quanto ci sono le loro astrazioni**.

I dettagli della realizzazione di una classe sono normalmente nascosti.

Ogni classe ha una doppia componente

1. **Una componente statica**, ossia i dati, costituita da campi o attributi dotati di nome, che contengono un valore. I campi caratterizzano lo stato degli oggetti durante l'esecuzione del programma.

Questi si distinguono in base al loro ambito d'azione (**scope**):

- **Attributi d'istanza**: sono associati ad una istanza e hanno un tempo di vita pari a quello dell'istanza alla quale sono associati.
- **Attributi di classe**: sono associati alle classi e condivisi da tutte le istanze della classe. Il loro tempo di vita è lo stesso della classe.

Gli attributi di istanza contribuiscono a caratterizzare lo stato di ogni singolo oggetto, mentre gli attributi di classe contribuiscono a definire il fattore comune allo stato di tutti gli oggetti di una classe.

2. **Una componente dinamica**, ossia i metodi (o operazioni), che rappresentano il comportamento comune degli oggetti appartenenti alla classe, cioè i servizi che possono essere richiesti a un oggetto di una classe. I metodi manipolano gli attributi.

I metodi possono essere classificati in:

- **Metodi costruttori**: sono invocati per creare (istanziare) gli oggetti e inizializzarli
 - **Metodi di accesso**: restituiscono astrazioni significative dello stato di un oggetto
 - **Metodi di trasformazione**: modificano lo stato di un oggetto
 - **Metodi distruttori**: sono invocati quando si rimuovono gli oggetti dalla memoria
- I metodi di accesso e trasformazione possono essere distinti in:

1. **Metodi di istanza**: operano su almeno un attributo di istanza, pertanto possono essere invocati solo specificando l'istanza
2. **Metodi di classe**: operano esclusivamente su attributi di classe, pertanto possono essere invocati specificando la classe. Si possono invocare metodi di classe anche quando non è stato istanziato alcun oggetto per quella classe

In linea di principio, l'invocazione di un metodo di classe può avvenire anche specificando un oggetto (e non la classe), tuttavia ciò è sconsigliato perché non evidenzia il fatto che si manipolano solo attributi di classe. Non è invece possibile invocare un metodo di istanza sulla classe

Notazione UML

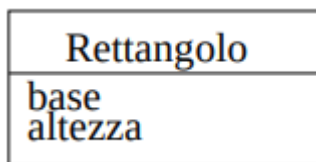
In UML una classe è resa graficamente mediante un rettangolo.



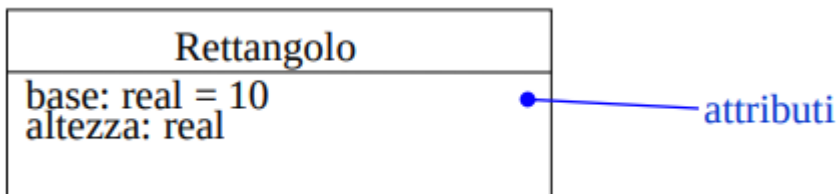
Ogni classe deve avere un nome che la contraddistingue dalle altre. Questo può essere semplice o indicare un percorso.



Graficamente gli attributi sono indicati sotto il nome della classe.

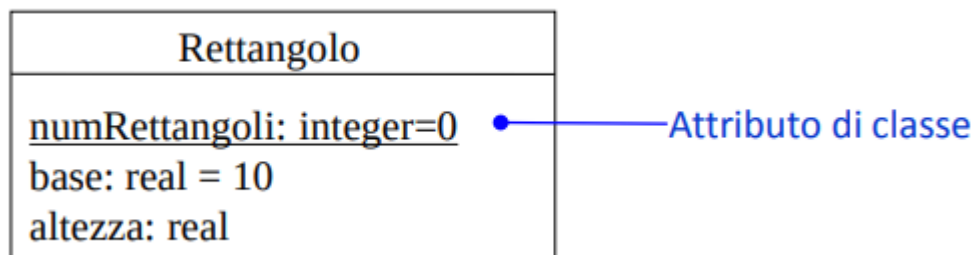


Di un attributo si può specificare l'insieme dei valori assunti (oggetti di una classe) e una eventuale inizializzazione



Attributi statici

Un attributo di classe (detto statico in UML) è indicato come sottolineato.

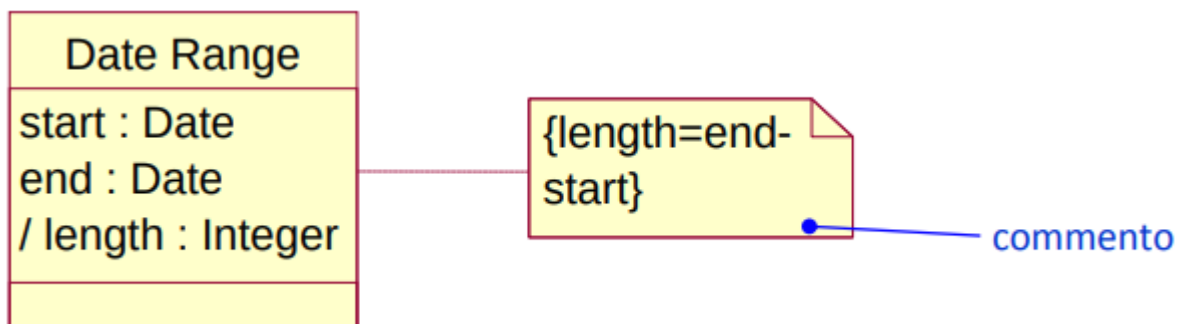


L'attributo numRettangoli indica il numero di oggetti della classe Rettangoli che sono stati istanziati. È un attributo statico (o di classe) in quanto condiviso da tutte le istanze della classe Rettangoli.

Attributi derivati

Gli attributi derivati sono quelli che possono essere calcolati partendo da altri attributi. UML prevede una rappresentazione specifica mediante una '/' (integer).

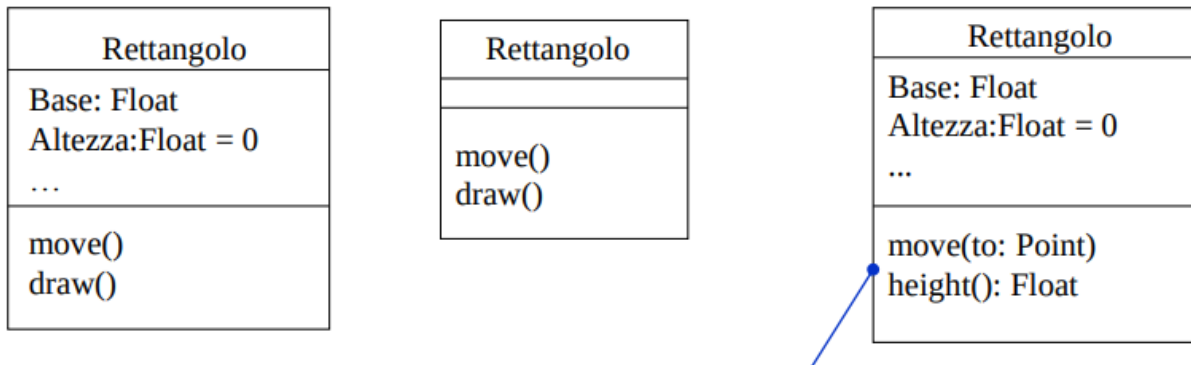
Si rappresenta con i commenti:



Il commento sulla destra fa parte della notazione standard UML e in questo caso è utilizzato per indicare come si calcola l'attributo derivato **length**. I commenti possono essere aggiunti a qualunque elemento della notazione UML.

Metodi

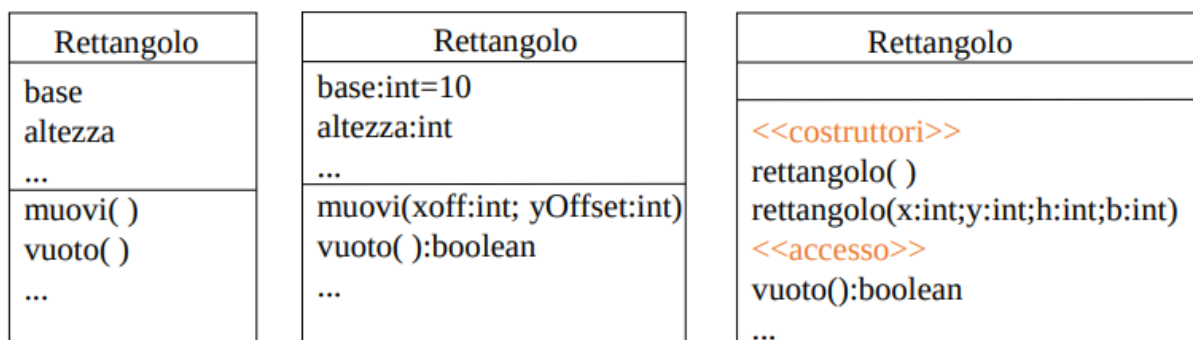
In UML le operazioni sono indicate graficamente in una sezione al di sotto degli attributi:



Di una operazione si può specificare la **segnatura**

Classi

Estendiamo la visione di classi con la specifica di attributi e metodi:



I diversi livelli di dettaglio consentono a chi progetta/modella di attribuire maggiore o minore importanza a determinati fattori a seconda della vista del sistema che si sta considerando

Stereotipi

Gli stereotipi sono dei tipici meccanismi di estendibilità di UML, essi estendono il vocabolario di UML, permettendo di creare nuovi blocchi per la costruzione dei modelli, derivandoli da blocchi già esistenti ma rendendoli specifici per il particolare dominio.

Gli stereotipi sono previsti dall'UML e sono rappresentati con le parentesi angolari «»

Visibilità delle classi

Gli attributi e i metodi di una classe possono avere diversi livelli di **visibilità**.

Un elemento (attributo o metodo) ha

- **Visibilità pubblica** quando può essere visto (utilizzato, invocato) da altre classi.
 - **Visibilità privata** quando può essere visto solo dalla classe di appartenenza
- Ad esempio, un metodo pubblico può essere invocato da qualunque punto del codice (purché la classe sia 'importata' in qualche modo), mentre un metodo privato può essere invocato solo da altri metodi della stessa classe

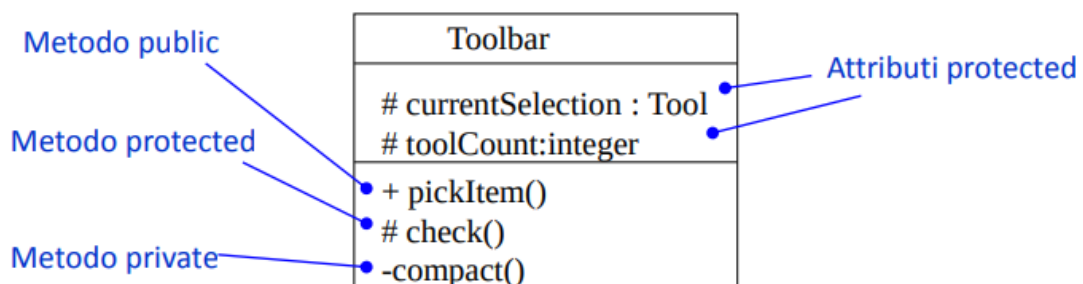
Altri livelli di visibilità sono:

- **Protetta:** l'elemento è visibile all'interno del package e all'esterno solo ai discendenti della classe di appartenenza
- **package:** l'elemento è visibile solo agli elementi del package che contiene la classe in cui l'elemento è definito

Visibilità degli elementi in UML

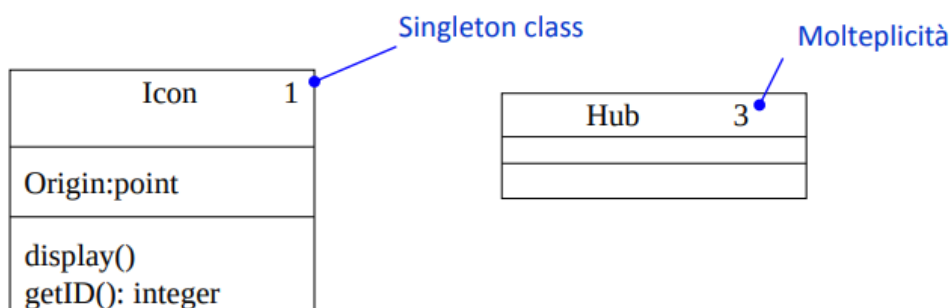
UML consente di specificare i livelli di visibilità di attributi e metodi utilizzando la seguente notazione:

- **public:** l'elemento è preceduto da un +
- **protected:** l'elemento è preceduto da un #
- **private:** l'elemento è preceduto da un -
- **package:** l'elemento è preceduto da un ~



Molteplicità della classe

Con **molteplicità di classe** si intende il numero di istanze che essa può avere. Generalmente non si pone un limite, tuttavia in alcuni casi è necessario indicare che la classe può avere una sola istanza (classe singoletto, singleton class) o comunque un numero ben definito di istanze. UML impone che tale valore sia indicato in alto a destra nell'icona rappresentante la classe:



22

Schema per la definizione di un attributo di una classe

[visibilità] nome [molteplicità] [:tipo]

[= valore iniziale] [{proprietà}]

Esempi di definizioni lecite di attributo:

- | | |
|---------------------------|------------------------------|
| • origine | solo il nome |
| • + origine | visibilità e nome |
| • origine : point | nome e tipo |
| • testa : *elemento | nome e tipo complesso |
| • nome [0..1] : String | nome, molteplicità e tipo |
| • origine : Point = (0,0) | nome, tipo e valore iniziale |
| • id : Integer {frozen} | nome, tipo e proprietà |

35

Proprietà per gli attributi

In UML esistono tre proprietà predefinite che possono essere utilizzate con gli attributi:

- **changeable**: non vi sono restrizioni sulla modificabilità dell'attributo
- **addOnly**: per gli attributi con molteplicità maggiore di uno, i valori possono essere aggiunti, ma una volta creati, non possono più essere rimossi o modificati
- **frozen**: il valore dell'attributo non può essere modificato dopo che l'oggetto è stato inizializzato (viene soltanto inizializzato con quei dati)

Nel caso in cui la proprietà non viene specificata si sottintende che assume valore **changeable**

Schema per la definizione di un operazione

Ricordiamo che in UML si distingue **operazione** e **metodo**:

- Una **operazione** è un servizio che può essere richiesto alla classe
- Un **metodo** è un'implementazione del servizio

In un operazione ci possono essere più implementazioni, il metodo è l'implementazione dell'operazione

Schema definizione operazione

Esempi di definizioni lecite di operazione :

- | | |
|------------------------------|---------------------------------|
| • visualizza | solo il nome |
| • + visualizza | visibilità e nome |
| • set(n : Nome, s : String) | nome e parametri |
| • getID() : Integer | nome e tipo del dato restituito |
| • riparti() {guarded} | nome e proprietà |

Ogni parametro riportato nella segnatura prende la forma:

[direzione] nome : tipo [= valore iniziale]

La **direzione** può assumere uno dei seguenti valori:

- | | | |
|----------------|--|----|
| • in | parametro di input, non può essere modificato | |
| • out | parametro di output, può essere modificato per comunicare un'informazione al chiamante | |
| • inout | parametro di input che comunque può essere modificato | 38 |

È possibile nel passaggio di parametri avere un valore di default

Per le operazioni possiamo specificare diverse proprietà:

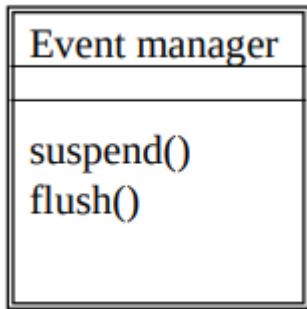
- **isQuery**: l'esecuzione dell'operazione lascia lo stato del sistema immutato, un'operazione con tale proprietà è quindi priva di **side-effect**
- **leaf**: l'operazione non può essere più specializzata (overriding) nelle sottoclassi (vedi final in Java)
- **sequential**: i chiamanti (callers) di questo oggetto devono coordinarsi affinché solo uno alla volta richieda il servizio. Nel caso di sovrapposizione la semantica e l'integrità dell'oggetto **non sono garantite**, potrebbe corrompersi
- **guarded**: simile al caso precedente, in ogni istante un solo chiamante può usufruire del servizio, tuttavia, in questo caso, la sequenzialità del servizio è gestita dalla classe proprietaria del servizio stesso (vedi synchronized in Java)
- **concurrent**: la semantica e l'integrità dell'oggetto è garantita anche in caso di chiamate multiple

Classe attiva

Un oggetto è **attivo** quando può generare thread concorrenti, una classe è attiva quando le sue istanze sono concorrenti con le altre.

In UML vengono rappresentati così

Rappresentazione classe attiva



Classi template

Una **classe template** definisce una famiglia di classi parametrizzate (con parametro di tipo). Non è possibile usare direttamente una classe template. È necessario prima specificare il tipo (operazione di istanziiazione). In Java, una classe template corrisponde a una classe generica.

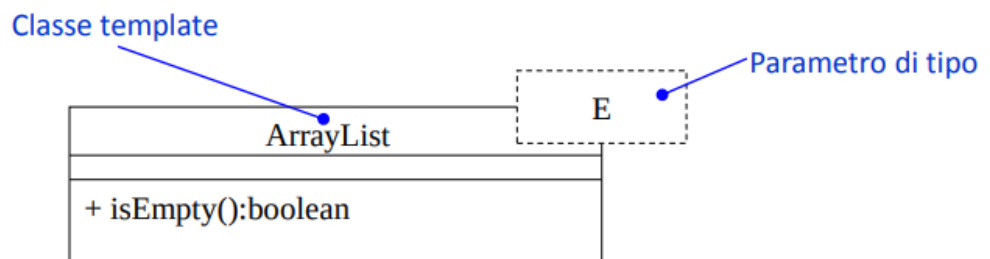
Queste classi template usano a generalizzare

📄 Rappresentazione UML

Esempio:

```
public class ArrayList<E>
```

Notazione UML



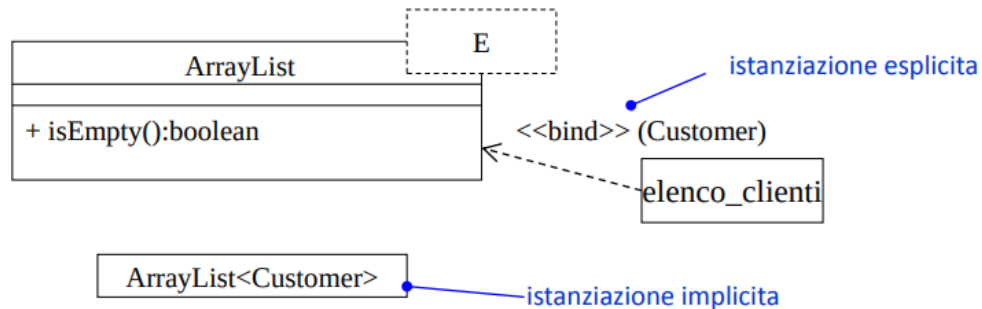
☰ Esempio di classe template

Si potrebbe voler istanziare questo template per creare una lista di clienti.

```
ArrayList<Customer> elenco_clienti;
```

L'istanziamento di una classe template può essere effettuata in due modi:

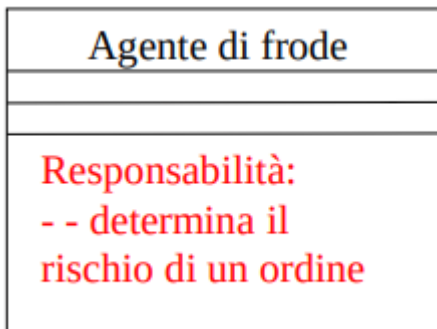
1. Implicitamente, dichiarando una classe il cui nome esplicita i parametri
2. Esplicitamente, mediante una dipendenza stereotipata <<bind>>



Individuazione delle responsabilità

In una buona modellazione OO di un sistema software è necessario stabilire le responsabilità da attribuire a ciascuna classe individuata, con attributi e operazioni che ha. UML consente di modellare le responsabilità in due modi:

- Specificandole all'interno della classe:

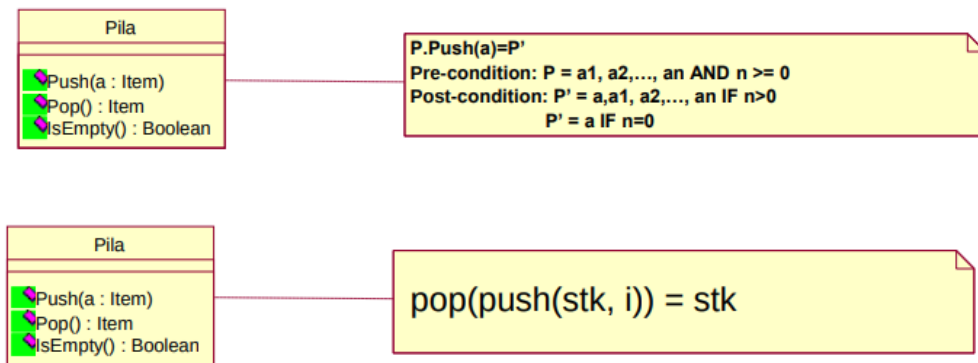


- Utilizzando delle note:



La specifica della responsabilità di una classe serve a definire cosa fa una classe (la semantica della classe), e in alternativa si possono usare le varie specifiche assiomatiche o

algebriche (sono consigliate quelle algebriche per la progettazione)



44

Al fine di rendere il sistema altamente riutilizzabile, il lavoro di individuazione delle classi deve essere condotto in modo preciso, è necessario a tal fine seguire una metodologia ben definita:

1. Identificare gli elementi che gli utenti usano per descrivere il problema
2. Per ogni astrazione individuata è necessario identificare un insieme di responsabilità, sincerandosi che vi sia un buon bilanciamento di responsabilità tra le classi
3. Fornire ad ogni classe gli attributi e le operazioni di cui ha bisogno per eseguire tali responsabilità

L'individuazione delle classi dipende dunque sia dalla realtà che si vuole modellare, sia dalla necessità di individuare il giusto bilanciamento dei compiti da assegnare, al fine di rendere il sistema software altamente mantenibile e riutilizzabile

Stereotipi di classi

Alcune metodologie suggeriscono l'individuazione e la classificazione delle classi secondo i seguenti gruppi:

- Classi entità: servono per la modellazione di dati
- Classi di confine: usate per l'interazione tra vari classi
- Classi di controllo: sono le classi che permettono i calcoli e operazioni sulle entità, quindi modificano i dati

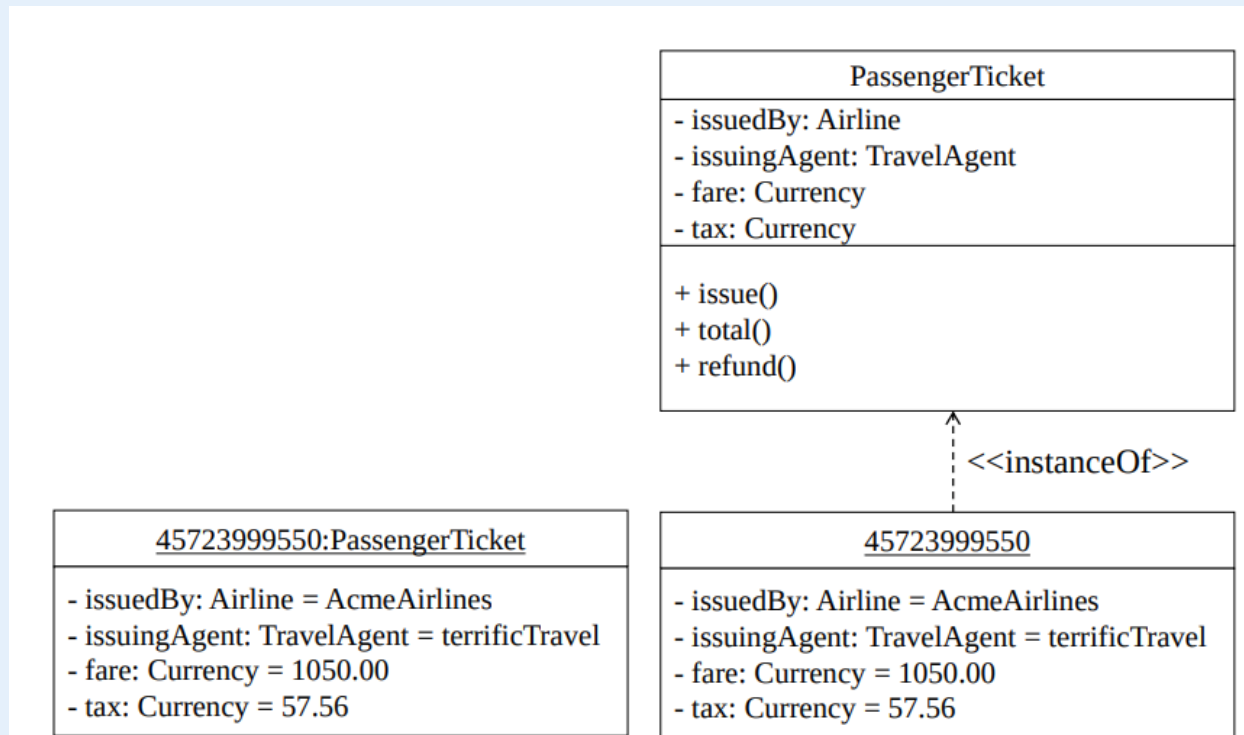
Tale suddivisione, modellata mediante l'utilizzo di stereotipi, permette il partizionamento del sistema in tre componenti differenti:

- Dominio
- Vista
- Controllo

Relazione Instance-of

Fra un oggetto e una classe sussiste una relazione `instance of` che specifica che un oggetto è una istanza di una classe, in UML sono rappresentate in questa maniera

❗ Rappresentazione Instance-of



Ereditarietà

Nella progettazione e programmazione OO una relazione fondamentale è quella esistente fra le classi: **la relazione di ereditarietà** (inheritance).

Una classe è considerata come un repertorio di conoscenze a partire dal quale è possibile definire altre classi più specifiche, che completano le conoscenze della loro classe madre.

Una sottoclasse è dunque, una specializzazione della descrizione di una classe, detta la sua **superclasse**, della quale essa mutua (parte di) gli attributi e i metodi. Quando eredita una sottoclasse deve ereditare sempre le caratteristiche della superclasse

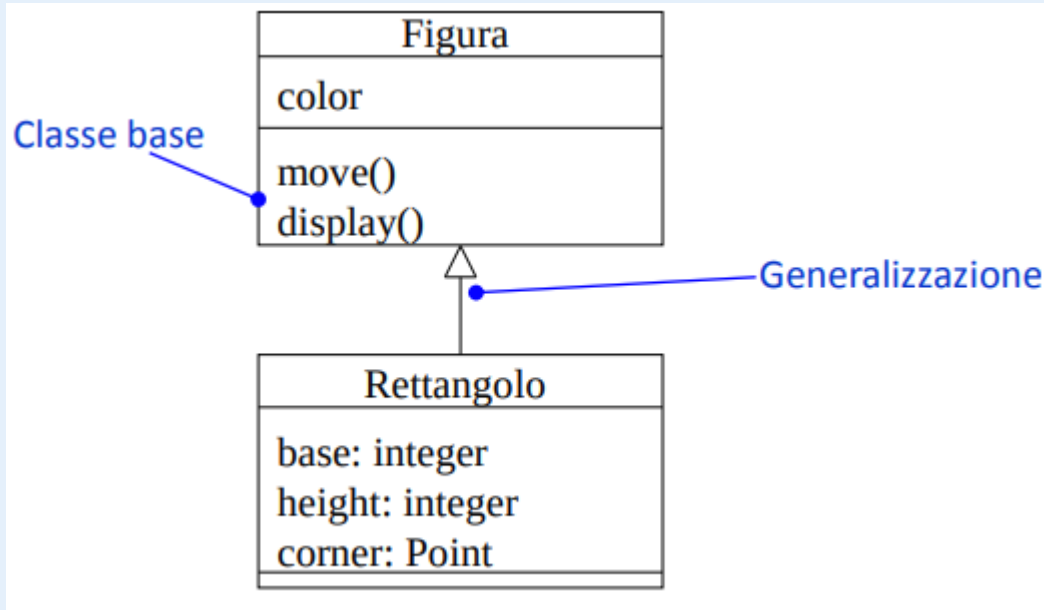
Ci sono diverse forme di ereditarietà:

Per estensione

In questo caso la sottoclasse introduce delle caratteristiche (attributi e metodi) non presenti nella superclasse e non applicabili a istanze della superclasse.

La visibilità (pubblica, protetta, privata, package) degli attributi e delle operazioni ereditate dalla superclasse non è modificata

❗ Rappresentazione UML



La rappresentazione della freccia è piena, non tratteggiata

⚠ Attenzione all'ereditarietà

Gli oggetti della sottoclasse hanno tutti quelli della superclasse, ma questo non vale nella maniera contraria!

Nell'esempio, `figura` non avrà mai l'attributo `base`

La specializzazione per estensione permette di sviluppare del codice estendibile, infatti se si dovesse avvertire la necessità di aggiungere ulteriori funzioni occorrerà soltanto individuare le classi interessate e derivare da queste nuove classi alle quali verranno aggiunti gli attributi e i metodi necessari per implementare le nuove funzioni

Per variazione funzionale

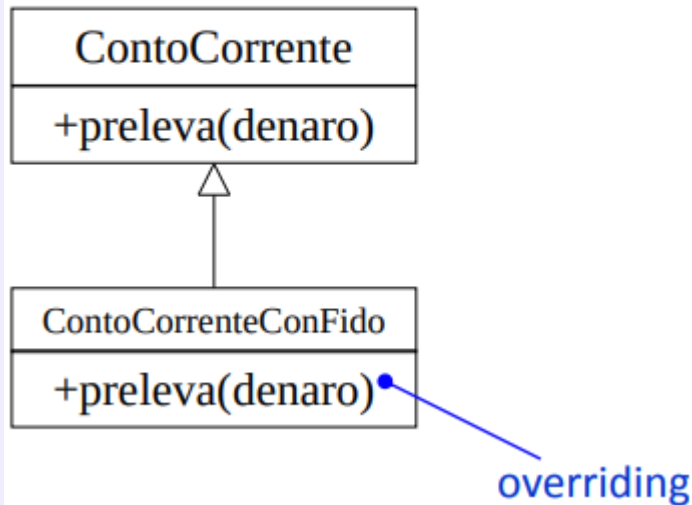
Nella variazione funzionale si ridefiniscono alcune caratteristiche (metodi) della superclasse quando quelle ereditate si rivelano inadeguate per l'insieme di oggetti descritti dalla sottoclasse, modificando il metodo ereditato.

La ridefinizione (**overriding**) del metodo ereditato **riguarda solo l'implementazione** e non la segnatura (quindi nome, parametri formali e parametro di ritorno).

Ogni richiesta di esecuzione del metodo ridefinito da parte di un oggetto della sottoclasse, farà riferimento alla nuova implementazione fornita nella sottoclasse

☰ Esempio di overriding

Esempio



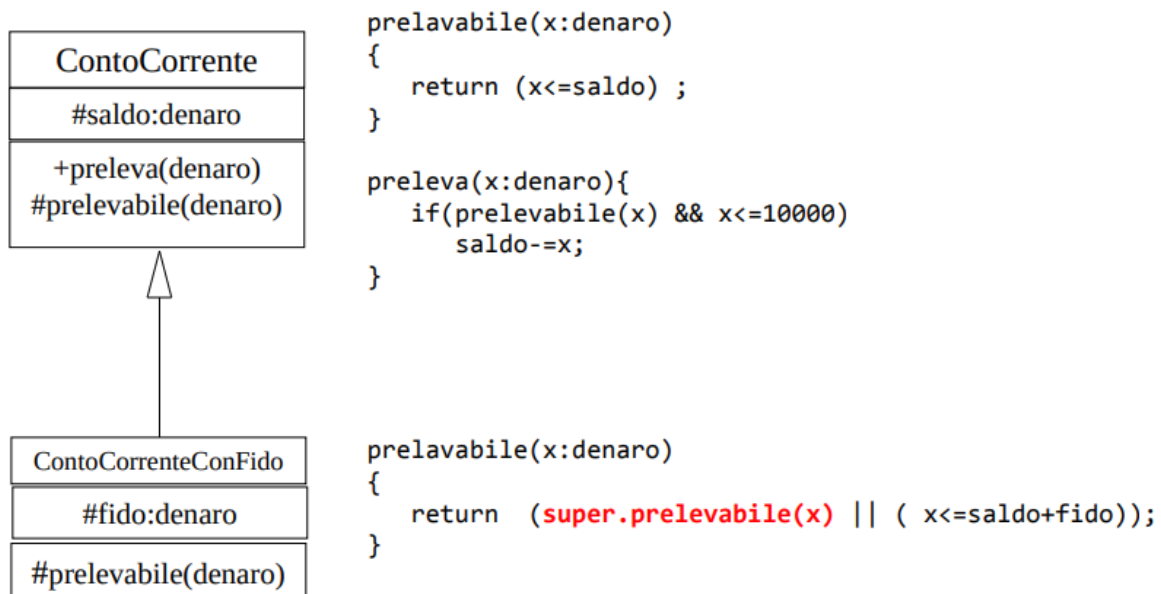
Nella classe `ContoCorrente` il metodo `preleva` controlla che il conto non vada in rosso

Il metodo `preleva` della classe `ContoCorrenteConFido` ridefinisce quello della superclasse per controllare che il prelievo non vada oltre il fido concesso

La ridefinizione non è incrementale, quindi i cambiamenti nel metodo originale devono essere riportati anche nei metodi ridefiniti, ma non esiste alcuna garanzia che questo accada e si possono introdurre degli errori.

Per mitigare gli effetti di questo problema, si può adottare qualche accorgimento nella realizzazione dei metodi per i quali si riconosce già in fase di progetto una incrementalità al cambiamento, ad esempio il metodo di una sottoclasse deve prevedere, nella sua realizzazione, l'invocazione del metodo della superclasse, e nel caso di risposta positiva, aggiungere ulteriori controlli

☰ Esempio di fix



Per combinare, in un overriding, il metodo della superclasse con il codice specifico della sottoclasse, i linguaggi di programmazione offrono diversi meccanismi

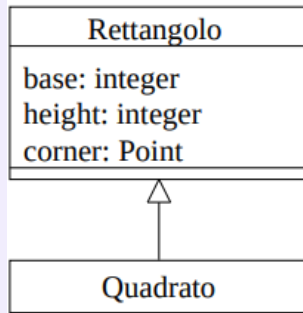
In Smalltalk e Java, un metodo definito in una (sotto-)classe può invocare una operazione della superclasse facendo riferimento a `super`, come `super.prelevabile()`.

Nell'ereditarietà per variazione funzionale, la visibilità (pubblica, protetta, privata, package) degli attributi e delle operazioni ereditate dalla superclasse non è modificata, inoltre la variazione funzionale attiene solo le operazioni di accesso e trasformazione di una classe, non andando a modificare i costruttori della superclasse originaria.

Per restrizione

Nella ereditarietà per restrizione, le istanze di una sottoclasse soddisfano vincoli che non sono necessariamente soddisfatti da istanze della superclasse.

☰ Esempio per restrizione



Esempio

In matematica è possibile trovare molti esempi. Un quadrato è un particolare caso di rettangolo nel quale i due lati hanno medesima lunghezza.

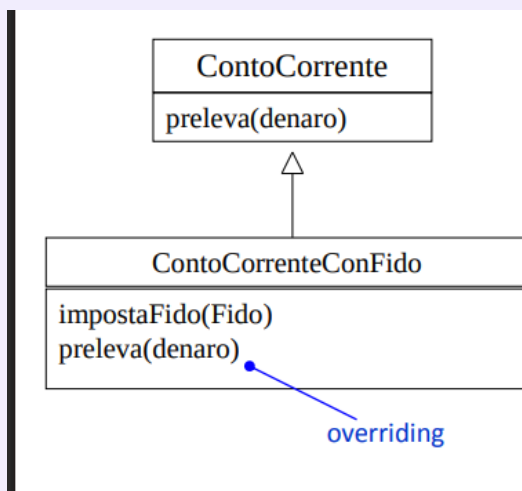
L'ereditarietà per restrizione non chiede la modifica della visibilità degli attributi e delle signature delle operazioni ereditate dalla superclasse.

58

Combinazioni di ereditarietà

Una classe può combinare diversi tipi di ereditarietà

≡ Esempio di ereditarietà combinata



In questo esempio si ha ereditarietà per estensione (si aggiunge il metodo *impostaFido*) e per variazione funzionale (si ridefinisce il metodo *preleva*).

Contents

Principio di sostituibilità (o generalizzazione)

📄 Definizione di principio di sostituibilità (o generalizzazione)

Data una dichiarazione di una variabile o di un parametro il cui tipo è dichiarato come *X*, una qualunque istanza di una classe che è discendente di *X* può essere usato come valore effettivo senza violare la semantica della dichiarazione e il suo uso. In altri termini, **l'istanza di un discendente può essere sostituita all'istanza di un ascendente**

La conseguenza del principio di sostituibilità è che una sottoclasse non può rimuovere o rinunciare a proprietà/metodi della superclasse, altrimenti una istanza della sottoclasse non sarà sostituibile in una situazione in cui si dichiara l'uso di istanze della superclasse. In effetti, preservando la visibilità degli attributi e dei metodi ereditati, così come accade nelle tre forme di ereditarietà viste, si garantisce che gli oggetti della sottoclasse offrano quanto meno gli stessi servizi degli oggetti della superclasse (anche sei servizi potranno essere implementati diversamente, come accade nella variazione funzionale).

Pertanto il principio di sostituibilità (o polimorfismo di inclusione) è compatibile con l'ereditarietà per estensione, variazione funzionale e restrizione

⚠ Cosa dice quindi il principio di sostituibilità?

L'oggetto di una sottoclasse può essere usato come se fosse un istanza della superclasse

Nelle varie ereditarietà, la relazione di ereditarietà fra classi corrisponde a una relazione di **generalizzazione** (o "is_a"), perché ogni istanza di una classe derivata da una classe base va considerata come (è anche) una istanza della classe base. Ci sarebbe un tipo di ereditarietà che non rispetta questo principio

Ereditarietà di implementazione

In questa ereditarietà, la sottoclasse utilizza il codice della superclasse (definizioni di attributi e metodi) per implementare l'astrazione associata.

☰ Esempio di ereditarietà di implementazione

Esempio

Una classe *Vettore* mette a disposizione una rappresentazione di un vettore, un costruttore, un metodo di accesso (a un elemento sulla base di un indice) e uno di trasformazione (cambiamento di stato di un elemento di cui si conosce la posizione)

Una classe *Pila* può essere definita come sottoclasse di *Vettore*, in modo da potersi basare sulla rappresentazione del vettore per poter rappresentare una pila. Si dice che la realizzazione del dato astratto *Pila* è **delegata** a quella del *Vettore*. Se si cambia la rappresentazione del *Vettore* si cambia anche quella della *Pila*

62

Esempio (cont):

Grazie all'ereditarietà per implementazione, la realizzazione degli operatori *push*, *pop*, *top* previsti per una pila può basarsi sugli operatori messi a disposizione dalla classe *Vettore*

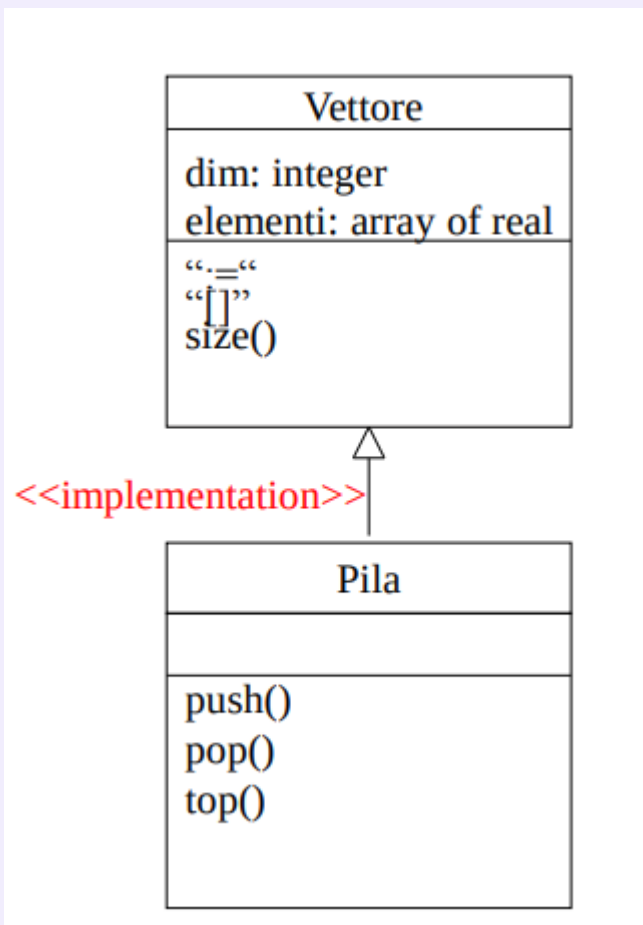
Si noti bene che la *Pila* ha un diverso insieme di operatori rispetto a quelli del *Vettore*. L'ereditarietà riguarda solo la realizzazione del vettore e l'implementazione dei metodi, **non l'interfaccia** della classe (ciò che è visibile agli utilizzatori di *Vettore*)

Pertanto **l'ereditarietà di implementazione comporta la modifica alla visibilità delle caratteristiche ereditate**

Questo tipo di ereditarietà è supportata su alcuni linguaggi di programmazione come C++ o Ada-95.

In UML l'ereditarietà di implementazione è indicata utilizzando lo stesso simbolo della generalizzazione, ma specificando a fianco lo stereotipo <<implementation>>.

Rappresentazione UML



L'ereditarietà di implementazione quindi non è compatibile con il principio di sostituibilità.
L'ereditarietà di implementazione permette un riutilizzo parziale del codice, ma non è consigliata.

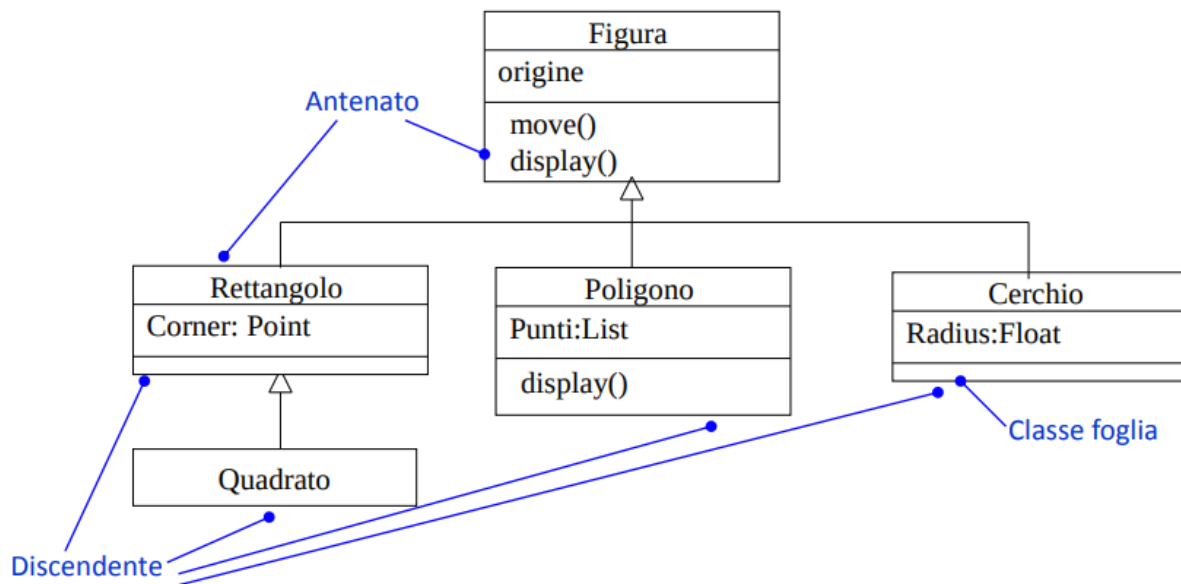
Proprietà della relazione di generalizzazione

La relazione di generalizzazione è **transitiva e antisimmetrica**, ossia:

- Comporta che le caratteristiche delle classi superiori sono ereditate dalle classi inferiori
- Definisce una direzione di attraversamento del grafo di ereditarietà che porta dalla sottoclasse alla superclasse

Grafo di ereditarietà

La rappresentazione della relazione di generalizzazione fra un insieme di classi definisce un **grafo di ereditarietà che è un grafo orientato aciclico**



Le classi foglia sono classi dove nessun'altra classe eredita.

Ereditarietà singola

Nell'ereditarietà singola, o semplice, ogni classe possiede una sola superclasse diretta, rendendo di fatto il grafo di ereditarietà un vero e proprio albero.

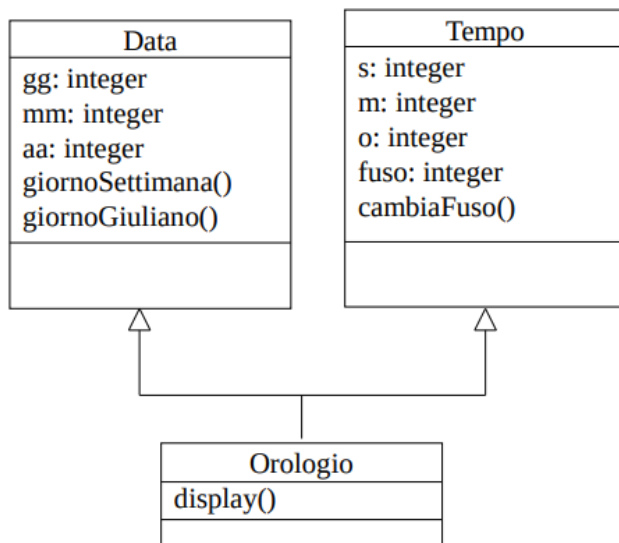
Il grafo di ereditarietà per una specifica classe si presenta come una catena lineare di antenati, dove gli elementi sono ordinati secondo una chiara relazione di ordine totale (come quello che vediamo sopra).

Quando il sistema deve determinare da quale classe venga ereditato un certo metodo (poniamo un metodo m), la risoluzione del problema è molto lineare. Il procedimento prevede prima la determinazione della catena degli antenati della classe e, successivamente, la ricerca della prima occorrenza della definizione o ridefinizione del metodo m , partendo dall'estremità inferiore della catena

Ereditarietà multipla

Quando una classe ha più superclassi si parla di **ereditarietà multipla (multiple inheritance)**

≡ Esempio di ereditarietà multipla



Esempio: la classe Orologio necessita dei servizi messi a disposizione sia da *Data* che da *Tempo*. In più ne implementa degli altri.

Il grafo di ereditarietà non è più un albero. Data una classe C, G(C) non è più una catena ma un grafo aciclico orientato. L'ordine fra le classi in G(C) è parziale.

71

L'ereditarietà multipla introduce notevoli difficoltà concettuali e tecniche, specialmente quando un metodo è definito in diverse superclassi ed entrambe le definizioni risultano ereditabili per la sottoclasse, generando così un conflitto, infatti se due o più classi padre definiscono lo stesso metodo, le diverse linearizzazioni possibili del grafo (ad esempio valutare prima un ramo rispetto a un altro) non aiutano il compilatore a scegliere in modo univoco quale metodo debba essere effettivamente ereditato.

Per cercare di gestire queste situazioni esistono dei criteri euristici:

1. Il primo di questi criteri si basa sulla "molteplicità dell'ereditarietà" e utilizza l'ordine logico in cui le superclassi sono state elencate al momento della dichiarazione per preferire una specifica linearizzazione del grafo rispetto a un'altra. Questo principio può essere in contraddizione con quello che indica di preferire l'eredità da classi più specifiche

Esempio: $A \multimap B, C$ e $C \multimap B, D$

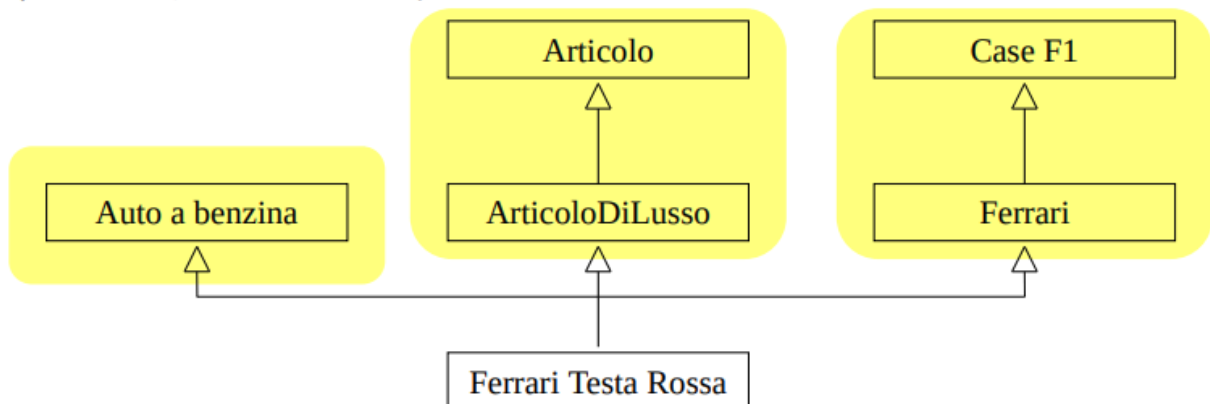
Se m è definito sia in B e sia in C , si dovrebbe preferire C in quanto C precede sempre B nelle linearizzazioni:

$A \rightarrow C \rightarrow B \rightarrow D$, però nella definizione di A abbiamo preferito B

2. Il secondo criterio fa leva sul concetto di "modularità", proponendo di scomporre il grafo di ereditarietà in sottomoduli separati che rappresentano diversi punti di vista concettuali sull'oggetto. In questo caso, le linearizzazioni non devono mai mescolare i sottografi

associati a moduli diversi, sebbene anche questo approccio

Esempio: i diversi moduli corrispondono ai diversi punti di vista (tipo alimentazione, tipo articolo, casa costruttrice).

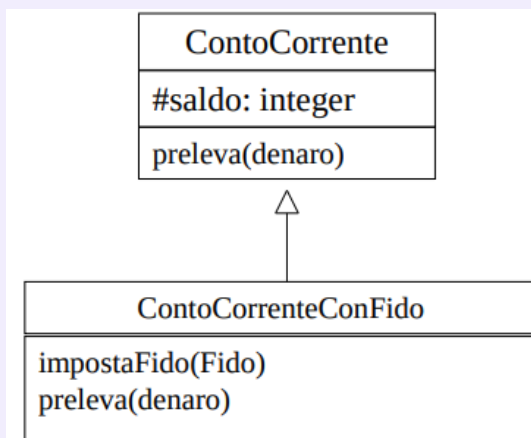


Visibilità protetta

La relazione di ereditarietà nel paradigma orientato agli oggetti introduce un livello di visibilità specifico e intermedio tra quello pubblico e quello privato, denominato **visibilità protetta (protected)**

Quando una caratteristica di una classe, che sia essa un attributo o un metodo, viene dichiarata con visibilità protetta, diviene accessibile non solo all'interno della classe stessa e a tutte le altre classi appartenenti al medesimo package, ma anche a tutte le sue classi discendenti

≡ Esempio di visibilità protetta



In questo esempio la classe *ContoCorrenteConFido* potrà accedere al dato saldo, mentre ciò non sarà permesso ad altre classi dislocate in altri package che non ereditano da *ContoCorrente*.

Classi astratte

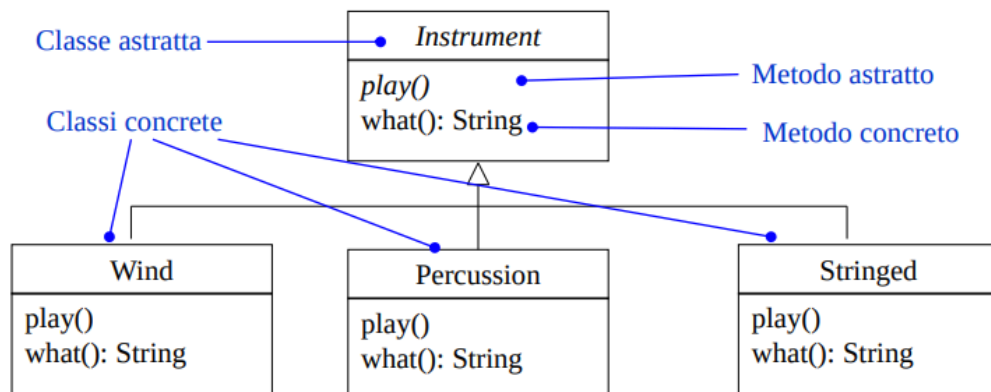
È possibile che esistano classi per le quali non è possibile generare delle istanze (**classi astratte**).

Una classe astratta può essere una classe non completamente specificata, in particolare non è definito il metodo corrispondente a una operazione.

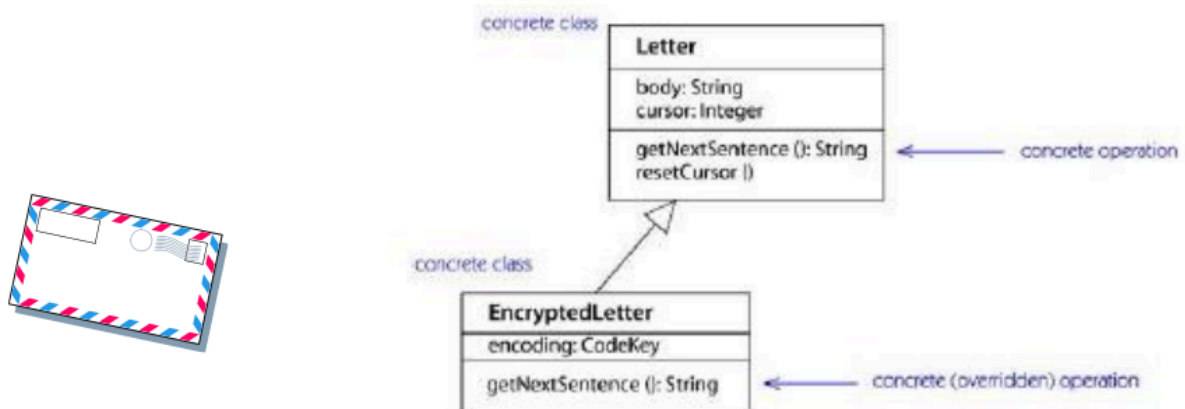
Le classi astratte non possono essere richiamate e non vi si potrà mai creare oggetti a partire da queste ultime.

Esempio di classe astratta

Classi astratte: notazione



Esempio: la classe degli strumenti musicali è astratta perché non possiamo definire il metodo *play* (non sappiamo come si suoni uno strumento in generale). Il metodo *play* è implementato nelle sottoclassi degli strumenti a fiato, degli strumenti a percussione, e dei cordati. Il metodo *what* è invece **concreto** (è implementato) e restituisce il nome della classe. Esso è ridefinito in ogni classe.



La classe *Letter* ha un metodo che restituisce la frase successiva da leggere e l'operazione *resetCursor* che riporta all'inizio del testo. La sottoclasse *EncryptedLetter* rappresenta una lettera crittografata. Il metodo *getNextSentence* è stato sovrascritto perché il testo dev'essere decrittato prima di essere restituito. L'implementazione dell'operazione *getNextSentence* è completamente diversa da quella ereditata.

PROBLEMA: Il progettista di *EncryptedLetter* non ha alcuna informazione per comprendere quale metodo di *Letter* dev'essere sovrascritto.

Le classi astratte sono strumenti per fattorizzare proprietà comuni tra classi simili e poterle organizzare in una gerarchia di ereditarietà, ossia fungono da **serbatoi di ereditarietà**, mettendo insieme diverse classi con proprietà uguali e comuni ma implementazioni diverse

Classi final (o foglia)

Una classe è detta **finale (final, o foglia)** quando non può essere ulteriormente specializzata, e quindi non può essere modificata. Da queste classi non è possibile ereditare, ed è fatto per ragioni di affidabilità.

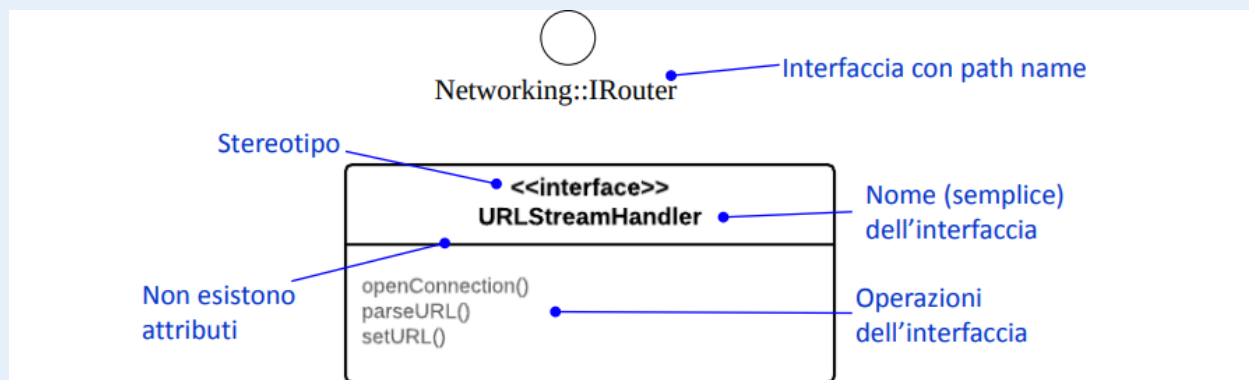
La dichiarazione di una classe foglia permette anche la generazione di codice ottimizzato in quanto facilita l'espansione in linea del codice (impossibile nel caso di metodi sovrascrivibili nelle sottoclassi)

Interfacce

Un'interfaccia è una collezione di metodi senza implementazione e attributi (ma può contenerne statici).

Una interfaccia è simile a una classe astratta i cui metodi sono tutti astratti e non dispone di attributi.

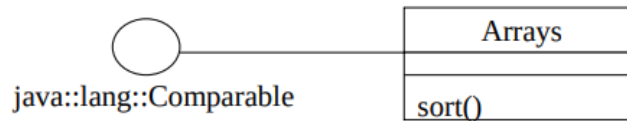
Rappresentazione UML interfacce



Le interfacce servono a disaccoppiare la definizione delle operazioni dalla loro implementazione. Per poter usare un certo oggetto è sufficiente conoscere la sua interfaccia: non serve conoscere l'implementazione

Esempio interfaccia

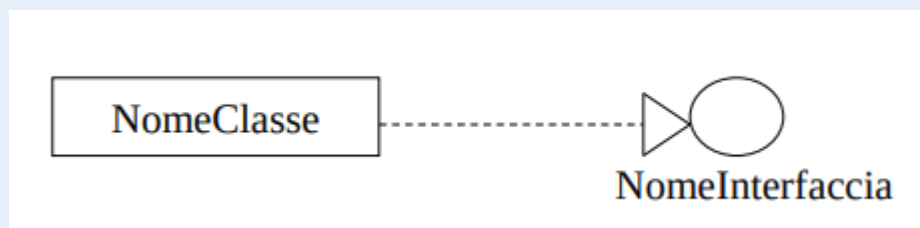
Ad esempio, il metodo *sort* della classe *Arrays*, che ordina un array di oggetti, necessita solo di sapere che gli elementi dell'array offrono il servizio *compareTo*, in modo da poterli confrontare. Non serve sapere "come" si realizza il servizio, cioè come sono effettivamente confrontati gli oggetti. Si crea così una **dipendenza** di implementazione fra la classe *Arrays* e l'interfaccia *Comparable*.



Relazione di realizzazione

Una o più classi possono realizzare/implementare le operazioni indicate in una interfaccia. La relazione che si stabilisce fra una interfaccia e una classe che la implementa è detta **relazione di realizzazione**. In UML è indicata con una freccia tratteggiata

Rappresentazione UML relazione di realizzazione



La relazione di realizzazione si presenta come un valido strumento per scindere la specifica di un "contratto" (cosa una classe deve implementare) e la sua implementazione (come si rendono i servizi del contratto)

Esempio di relazione di realizzazione

Esempio: l'interfaccia `java.lang.Comparable` specifica una sola operazione.

```
public int compareTo(Object o)
```

che prende un oggetto generico `Object` come argomento e restituisce un valore negativo se l'argomento è più piccolo dell'oggetto corrente, zero se è uguale e un valore positivo se è maggiore.

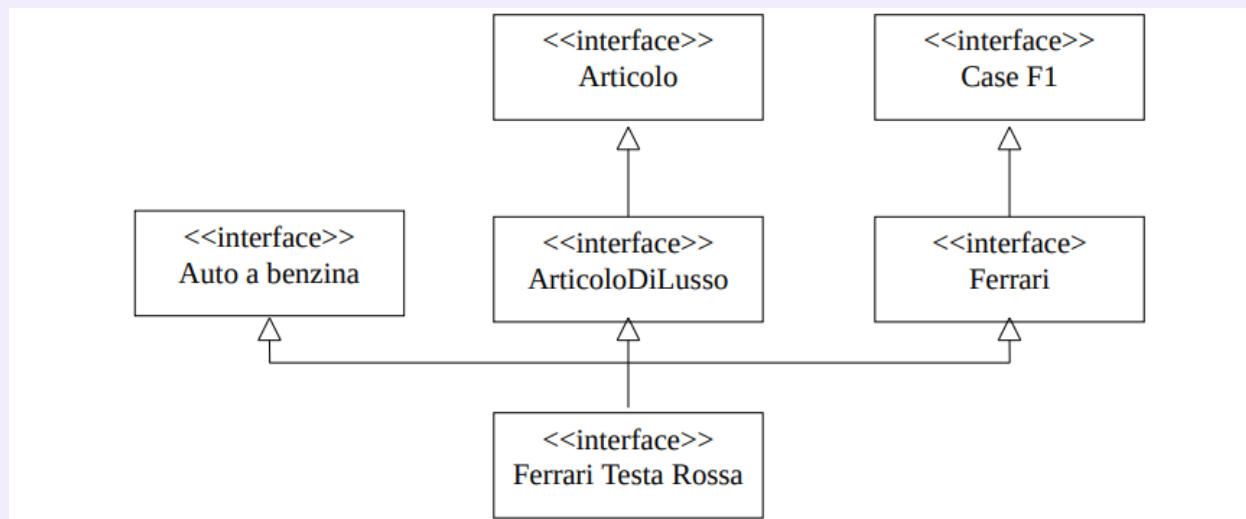
Una qualunque classe che implementa questa interfaccia deve fornire una **implementazione** (un metodo) per `compareTo`.



Ereditarietà

Anche le interfacce possono ereditare da altre interfacce, poiché non ci sono implementazioni, la relazione di ereditarietà è naturalmente una relazione di generalizzazione “is_a”.

≡ Esempio di interfacce



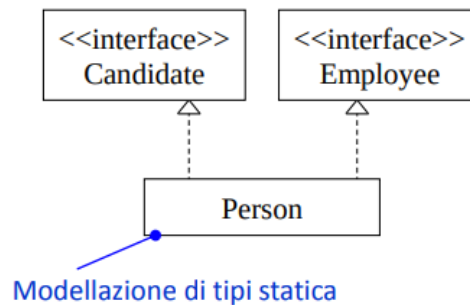
Poiché non si considerano le implementazioni delle operazioni, l'ereditarietà multipla su interfacce non pone problemi di conflitto di realizzazione, per questo alcuni linguaggi di programmazione, come Java, permettono l'ereditarietà singola sulle classi (in quanto specificano anche le implementazioni) mentre permettono l'ereditarietà multipla sulle interfacce. Questo distingue le classi astratte dalle interfacce.

Relazione di più interfacce

Poiché non possono sorgere problemi di conflitto di realizzazione, è permesso a una classe di realizzare più interfacce, per di più non correlate da una relazione di generalizzazione.

≡ Esempio di relazione di più interfacce

Ad esempio, una classe *Person* può implementare due interfacce:

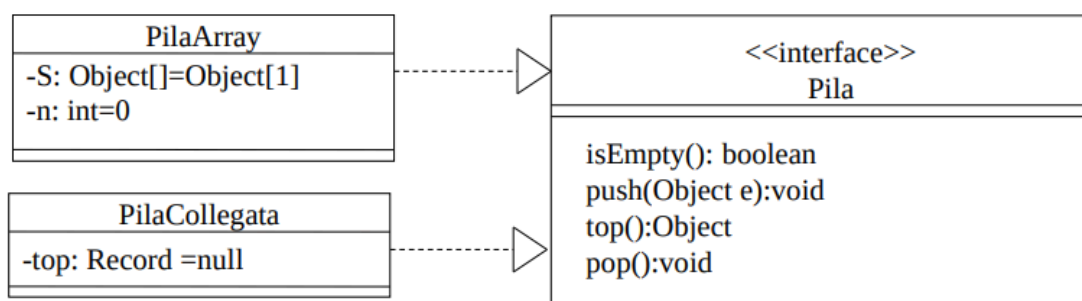


In questo modo tutto il codice che utilizza *Candidate* e *Employee* può essere utilizzato su oggetti di classe *Person*.

Realizzazioni multiple di una interfaccia

Più classi possono implementare la stessa interfaccia, è il caso una interfaccia definita per dati astratti molto utilizzati nello sviluppo del software, come pile, code, liste, alberi e grafi, per i quali sono possibili molteplici realizzazioni

≡ Esempio di realizzazione multiple di una interfaccia



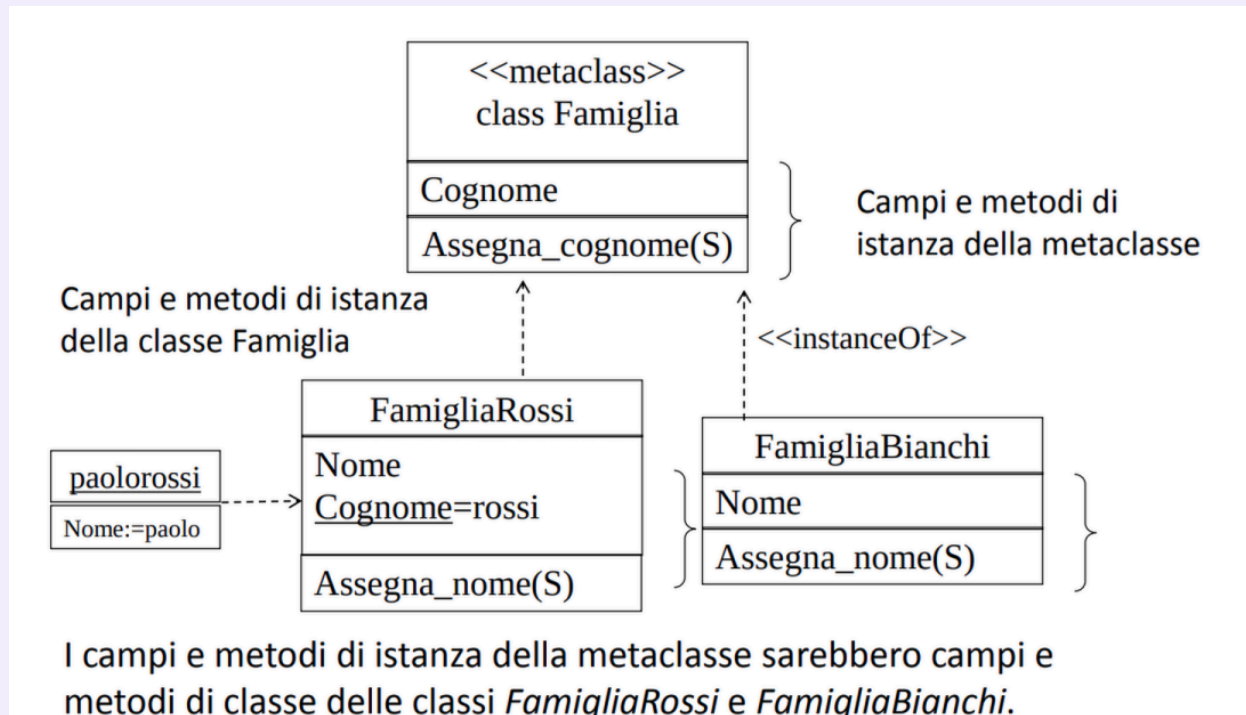
Il codice applicativo che intende fare uso di una pila **dichiarerà delle variabili di tipo *Pila***, svincolandosi dalla specifica realizzazione. Questa sarà specificata solo al **momento della inizializzazione** della variabile. Si garantisce così una forte invarianza ai cambiamenti delle realizzazioni di una pila.

95

Meta-classi

Una meta-classe è una classe particolare per descrivere la struttura (dati e operazioni) di altre classi

≡ Esempio di metaclassa



In Java il modello di metaclassa è differente da quello normale che troveremmo in C. Tutte le classi ereditano da `Object`, questa ha anche il metodo `getClass()` che permette di restituire per ogni oggetto un'istanza della classe `Class` che descrive la classe di appartenenza dell'oggetto.

`Class` è detta **metaclassa** ma non in senso stretto, poichè i suoi oggetti non sono classi ma le descrivono.

Questo modello serve per realizzare il meccanismo di **riflessione**, che permette ad un oggetto di stabilire a runtime le componenti di una classe

Classe "Class"

`Class` include metodi per:

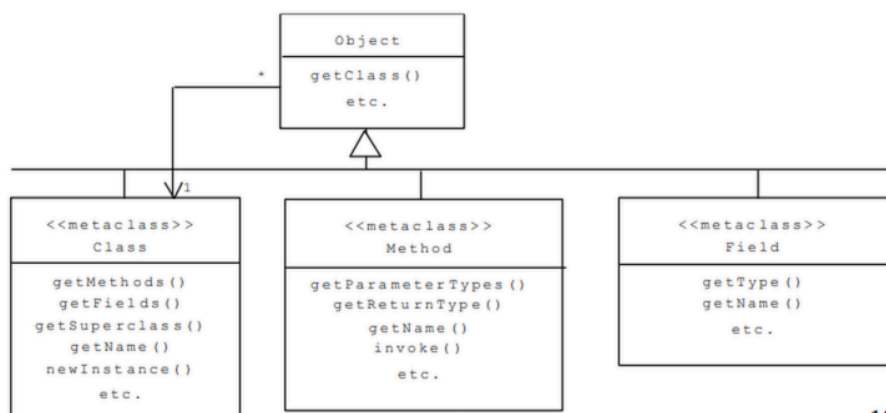
- Scoprire il nome della classe (`getName`)
- Scoprire il nome della superclasse (`getSuperClass`)
- Scoprire il nome dei metodi (`getMethods`)
- Scoprire il nome degli attributi (`getFields`)

Questi appartengono rispettivamente alle classi `Method` e `Field`. `Class`, `Method`, e `Field` sono sottoclassi di `Object` come tutte le classi.

Questo è utile, poichè la JVM non sa ovviamente quali sono le istanze, quindi a run-time le scopre e le utilizza.

📄 Rappresentazione UML delle metaclassi

<<metaclass>> è lo stereotipo usato in UML per indicare una metaclassa.



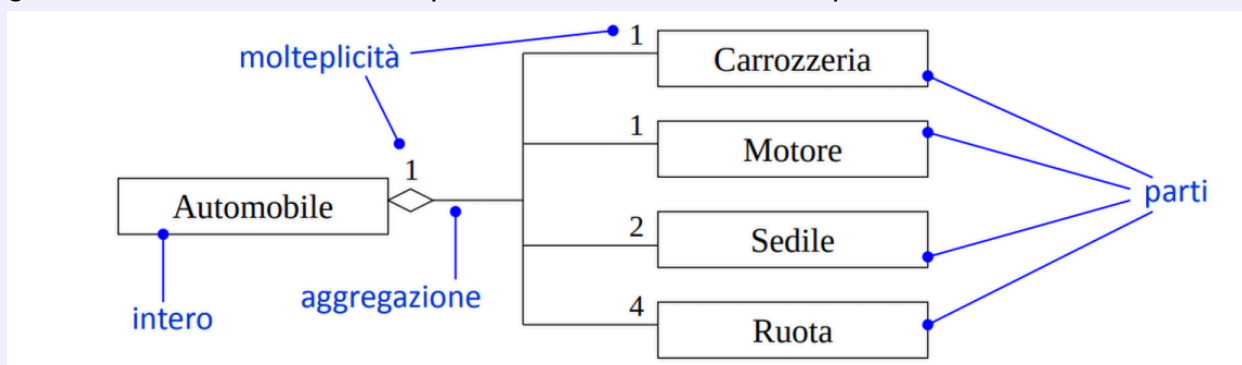
100

Aggregazione di oggetti

L'ereditarietà non è sempre lo strumento adatto per costruire oggetti, infatti spesso un oggetto è ottenuto **aggregando** altri oggetti

☰ Esempio di aggregazione

Definire una classe `Automobile` tramite ereditarietà multipla da `Motore` o `Ruota` è un errore concettuale, poiché l'ereditarietà fonde i comportamenti anziché comporli. Un'automobile è composta da parti, ma il suo comportamento non è la semplice unione di quello dei componenti. Inoltre, l'ereditarietà di implementazione non permetterebbe di gestire correttamente la molteplicità, come nel caso delle quattro ruote di un'auto.



La molteplicità consente di indicare quanti oggetti possono essere aggregati

L'uso dell'aggregazione viene suggerito nelle seguenti situazioni:

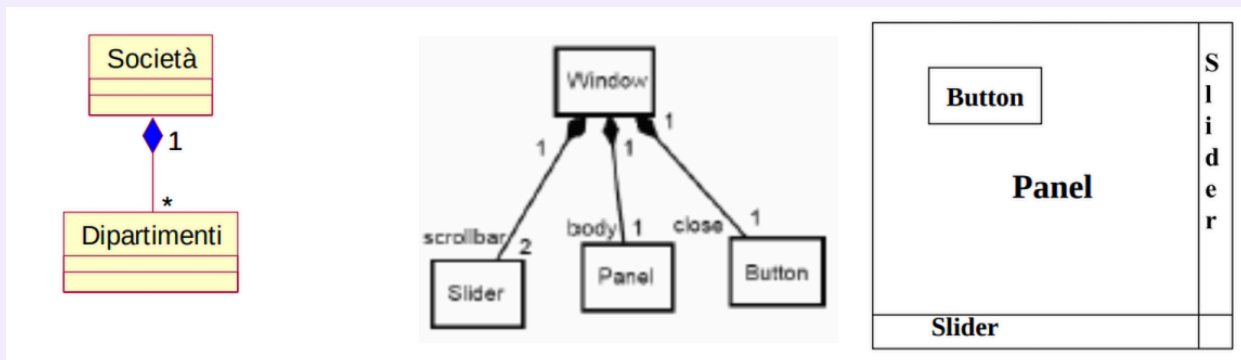
1. Contenimento fisico (pagina di un libro),
2. Appartenenza (giocatore in una squadra),
3. Composizione funzionale (ruote di un'auto)

Si possono stabilire legami con più istanze di una classe che descrive un componente (per esempio, più ruote). La relazione che si stabilisce in questo modo fra le classi è detta di

aggregazione o composizione (o relazione “has_a”).

L'aggregazione è un'associazione debole, se l'intero viene distrutto, le parti possono sopravvivere, mentre la composizione è un'associazione forte che implica una dipendenza esistenziale: le parti nascono e muoiono con il contenitore e non possono essere condivise con altri oggetti.

☰ Esempio di composizione



La composizione viene indicata con un rombo pieno in UML

Ereditarietà vs. Aggregazione

La scelta tra queste due relazioni non è sempre immediata. Se consideriamo una classe `Orologio` che deve gestire `Data` e `Tempo`, possiamo usare l'ereditarietà multipla o comporre l'oggetto con due campi specifici:

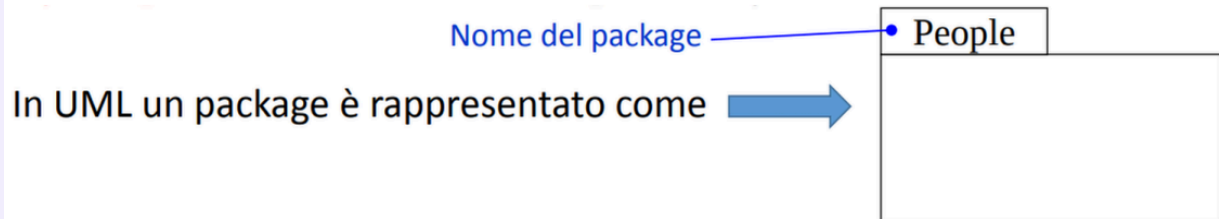
- Nell'ereditarietà, l'oggetto accede direttamente ai campi delle superclassi e beneficia del polimorfismo di inclusione ("is_a").
- Nell'aggregazione, l'accesso richiede l'invocazione di metodi sui componenti. In generale, l'aggregazione si usa quando servono i servizi di una classe ma non la sua interfaccia, mentre l'ereditarietà permette di riutilizzare codice polimorfo pre-esistente.

Il meccanismo di aggregazione/composizione è generalmente usato quando si vogliono utilizzare i servizi di una classe predefinita ma non la sua interfaccia. L'ereditarietà di implementazione, qualora non dovesse essere permessa da un linguaggio di programmazione, potrebbe essere resa da una relazione di aggregazione/composizione.

Organizzazione in Package

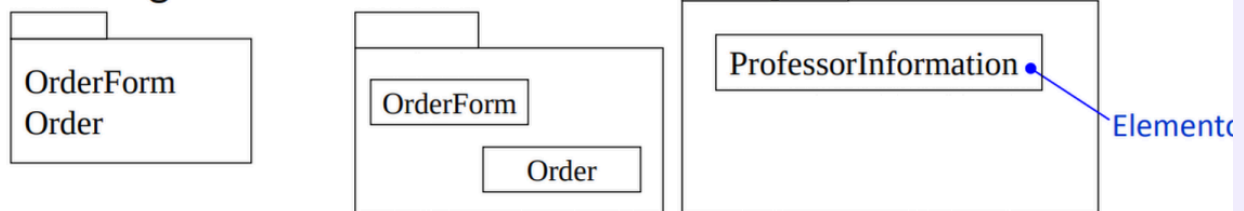
Per gestire la complessità, le classi vengono organizzate in **package**, ossia un meccanismo generale per organizzare le classi in gruppi

☰ Rappresentazione UML dei package



In genere i package si usano per riunire classi, ma nella notazione UML essi possono includere qualsiasi costrutto UML e possono essere persino eterogenei (ad esempio, possono contenere classi e interfacce).

È consentito mostrare il contenuto di un package sia testualmente sia graficamente:



Un package definisce un namespace (spazio degli identificatori) per i suoi elementi. Questo significa che ogni classe di un package di classi deve avere un nome distinto all'interno del package che la racchiude. Il nome completo (o qualificato) usa la notazione `::`.

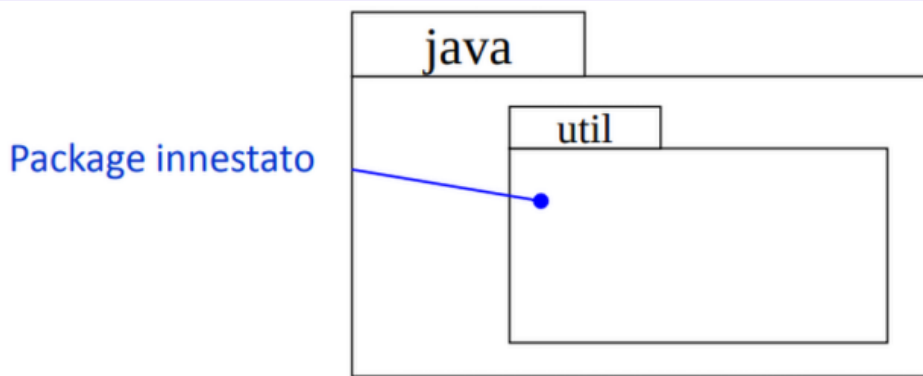
I package possono importare elementi altrui per evitare nomi qualificati, ma tale relazione non è transitiva.

≡ Esempio di relazione non transitiva

Se A importa B e B importa C, allora A può usare (senza qualificarli) gli elementi di B e B può usare (senza qualificarli) gli elementi di C, ma non è detto che A possa usare (senza qualificarli) gli elementi di C (a meno che A non importi anche C).

I package possono essere innestati senza alcun limite di profondità

≡ Esempio di package innestato

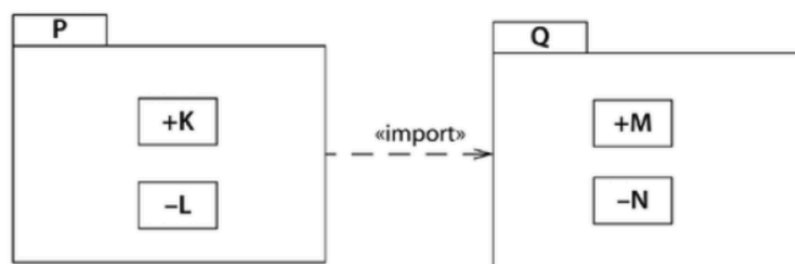


Si può anche specificare la visibilità degli elementi di un package:

- Public (+) sono visibili ad altri elementi del package stesso, a uno dei package innestati o a package che li importano.
- Private (-) non sono visibili all'esterno del package

≡ Esempio di package pubblico e privato

Esempio



Il package P importa il package Q. Le classi K e L nel package P possono usare M come nome di classe senza qualificarlo (::) poiché M è pubblica, ma non possono vedere la classe N, in quanto privata. Le classi M e N nel package Q possono usare il nome qualificato P::K per referenziare la classe pubblica K nel package P, ma non possono vedere in alcun modo la classe L.

122

Classi interne

Le **classi interne** (inner class) sono dichiarate dentro una classe ospite e possono essere private.

Esse possono accedere a tutti i membri della classe ospitante, mentre l'inverso è limitato alla parte pubblica.

Un'istanza della inner class non può esistere senza l'oggetto ospitante e non può contenere campi statici.

 **Attenzione**

In Java, una classe top level non può mai essere privata

Il polimorfismo

Il polimorfismo permette di associare diverse realizzazioni (o morfismi) a un'unica operazione

Il concetto di polimorfismo è stato definito informalmente da Christopher Strachey in due tipi principali:

- **Polimorfismo parametrico**: è ottenuto quando una funzione lavora uniformemente su una gamma di tipi. Questi tipi normalmente esibiscono una qualche struttura comune.
- **Polimorfismo ad-hoc**: è ottenuto quando una funzione lavora, o sembra lavorare, su tipi differenti (che potrebbero non esibire una struttura comune) e potrebbe comportarsi in modo totalmente differente per ciascuno di essi.

La classificazione di Cardelli e Wegner introduce una nuova forma di polimorfismo, quello **per inclusione**, al fine di modellare i concetti di sottotipo e di ereditarietà.

Il polimorfismo per inclusione e parametrico sono classificati come due sottocategorie del polimorfismo universale. L'idea di polimorfismo universale è quella di poter operare su un numero infinito di tipi, a patto che essi rispettino alcuni vincoli.

Polimorfismo ad-hoc

La coercizione

La coercizione è il meccanismo di conversione implicita operata da un compilatore per applicare un operatore definito per oggetti di tipo T1 anche a oggetti di tipo T2.

☰ Esempio di coercizione generale

Esempio:

$$3.14 + 5$$

L'operatore + è definito per valori reali, ma lo si può usare su un insieme di tipi più grande di quello per il quale è stato definito.

Senza coercizione, avremmo un errore di tipo.

☰ Esempio di coercizione in Java (unboxing e autounboxing)

```

int i;
Integer j;
i = 1;
j = 2;
i = j; // unboxing: Integer - int
j = i; // autoboxing: int - Integer

```

Le coercizioni possono essere stabilite staticamente al compile-time (inserendole automaticamente fra gli argomenti e le funzioni al momento della compilazione) oppure determinate dinamicamente a run-time.

La coercizione è la forma di polimorfismo più semplice, infatti essa opera a un livello semantico, cioè cambiando la rappresentazione del dato

Overloading

Si ha il polimorfismo per **overloading** quando lo stesso identificatore è usato per metodi differenti, disambiguati dal contesto o dal tipo degli argomenti

☰ Esempio di overloading su un operatore

```

// Overloading dell'operatore +
class Rational {
public:
    Rational(double);
    const Rational& operator+(const Rational& other);
    ...
};

// Overloading del nome di funzione max
double max(double d1, double d2);
char max(char c1, char c2);
char* max(char* s1, char* s2);
const char* max(const char* s1, const char* s2);

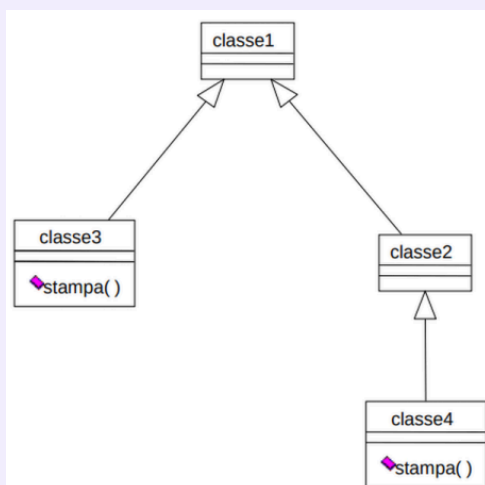
```

L'overloading può interagire con la coercizione: se `+` è definito per due interi e per due reali, espressioni miste come `3 + 4.0` non causano errori di tipo grazie alla combinazione dei due meccanismi.

Nel paradigma a oggetti si ha overloading anche tra funzioni con lo stesso nome definite in classi non correlate gerarchicamente: la disambiguazione si basa sulla classe dell'istanza su cui viene invocato il metodo.

Nel paradigma a oggetti si ha overloading anche nel caso di funzioni con medesimo nome ma definite in classi non correlate gerarchicamente

≡ Esempio polimorfismo meno potente



In questo esempio l'informazione di contesto utilizzata per disambiguare è la classe di appartenenza dell'istanza sulla quale si invoca il metodo stampa.

Questo polimorfismo è meno potente di quello universale, perché l'unicità del nome del metodo nasconde una molteplicità di implementazioni.

Polimorfismo parametrico

Nel polimorfismo parametrico, una funzione polimorfa ha un parametro di tipo esplicito o implicito, che determina il tipo dell'argomento per ciascuna applicazione della funzione.

Le funzioni che esibiscono il polimorfismo parametrico sono anche dette **funzioni generiche**.

Una funzione generica può lavorare su argomenti di molti tipi, generalmente esibendo lo stesso comportamento indipendentemente dal tipo dell'argomento.

Poliformismo per inclusione

Questo polimorfismo deriva dalla relazione tra sottotipi, dove un oggetto può appartenere a una classe e a tutte le sue superclassi.

Ciò permette di assegnare un'istanza di una sottoclasse a una variabile della superclasse

Esso si manifesta in due modi:

- Si può assegnare un oggetto di una qualsiasi sottoclasse di una classe C a una variabile definita di classe C
- Una funzione che opera su un oggetto di classe C può essere applicata anche a oggetti di classe C', sottoclasse di C.

Legame statico/dinamico

Nella maggior parte dei linguaggi di programmazione la visibilità degli identificatori e dei legami (binding) dei nomi alle dichiarazioni è determinata al momento della compilazione (compile-time), in questo caso si parla di ambito **d'azione statico (static scope)**.

Nell'ambito **d'azione dinamico (dynamic scope)**, il legame fra l'uso di un identificatore e la sua dichiarazione dipende dall'ordine di esecuzione, e così è differito al momento dell'esecuzione (run-time).